

1. Identificación de Pares: Enumera tres claves (keys) presentes en el nivel superior del objeto JSON.

Identificación de Pares: tres claves keys

"hostname" (valor: "R1-Core")

"uptime_days" (valor: 45)

"managed_by_apic" (valor: true)

2. Tipos de Datos: Identifica un ejemplo de cada uno de los siguientes tipos en el código: String, Number, Boolean y Null.

```
{
  "hostname": "R1-Core", : String
  "uptime_days": 45, : Numbers
  "managed_by_apic": true, : Boolean
  "interfaces": [ :
    {
      "name": "GigabitEthernet0/0",
      "description": "Link to WAN",
      "enabled": true,
      "mtu": 1500
    },
    {
      "name": "GigabitEthernet0/1",
      "description": null, : Null
      "enabled": false,
      "mtu": 1500
    }
  ]
}
```

Parte 2: Detección de Errores de Sintaxis

En la automatización de redes, un error de sintaxis en un payload JSON puede causar que la configuración de un dispositivo falle. Analiza el siguiente fragmento de código destinado a configurar una VLAN y responde.

```
{  
  "vlan_id": 10  
  "name": "Usuarios_Ventas"  
  "tagged": true,  
  "description": "VLAN para el departamento de ventas",  
}
```

3. ¿Qué errores de sintaxis impiden que este JSON sea válido según los estándares de red y programación?

Selecciona las dos opciones correctas.

A) El valor de 'vlan_id' debería estar entre comillas dobles obligatoriamente.

B) Las claves como 'vlan_id' no deben llevar guiones bajos.

C) Existe una coma final (trailing comma) después del valor de 'description'.

D) Faltan comas para separar los pares clave-valor después de 'vlan_id' y 'name'.

4. Reflexión Técnica: Explica por qué es crítico que un controlador como Cisco DNA Center reciba

Cisco DNA Center necesita recibir un JSON bien validado porque, si trae aunque sea una coma de más o de menos, el sistema no entiende la instrucción y simplemente no aplica el cambio. Lo más delicado es que en una red real la automatización trabaja sobre muchos equipos al mismo tiempo, así que un pequeño descuido puede convertirse en un problema grande: usuarios sin conexión, servicios caídos o configuraciones a medias.

Actividad 2: Análisis de Datos de Red con Python

Conceptos Clave

`json.loads()`: Función del módulo `json` de Python que convierte una cadena de texto en formato JSON a un diccionario o lista de Python.

Parsing: El proceso de analizar una estructura de datos para extraer información específica.

Parte 1: Análisis de Inventario

Imagina que estás automatizando la auditoría de una red empresarial. Has recibido el siguiente objeto JSON que representa el inventario actual de dispositivos. Tu objetivo es identificar qué equipos están fuera de línea (`down`) para generar un reporte de incidentes.

```
[
{"hostname": "Core-SW01", "ip": "10.0.0.1", "status": "up"},
{"hostname": "Dist-SW02", "ip": "10.0.0.2", "status": "down"},
{"hostname": "Access-SW03", "ip": "10.0.0.3", "status": "up"},
{"hostname": "Edge-R01", "ip": "172.16.1.1", "status": "down"}
]
```

1. Escribe el pseudocódigo o completa el script de Python necesario para procesar la variable `inventario_json` (que contiene el texto anterior). El script debe filtrar e imprimir únicamente los hostnames de los dispositivos cuyo estado sea `'down'`

```
inventario = [
    {"hostname": "Core-SW01", "ip": "10.0.0.1", "status": "up"},
    {"hostname": "Dist-SW02", "ip": "10.0.0.2", "status": "down"},
    {"hostname": "Access-SW03", "ip": "10.0.0.3", "status": "up"},
    {"hostname": "Edge-R01", "ip": "172.16.1.1", "status": "down"}
]
```

```
# Imprimir todos los hostnames
```

```
for h in inventario:
```

```
    print(h["hostname"])
```

```
#imprimir todas las ips
```

```
for h in inventario:
```

```
print(h["ip"])
```

```
#Imprimir todos los que esten en down
```

```
for h in inventario:
```

```
    if h["status"] == "down":
```

```
        print(h["hostname"], h["ip"])
```

Parte 2: Comparativa de Formatos (JSON vs. XML)

En redes modernas, REST APIs suelen preferir JSON por ser más ligero, pero muchos sistemas heredados o protocolos como NETCONF utilizan XML. Observa cómo se estructuran los mismos datos en ambos formatos.

Formato JSON	Formato XML
<pre>{ "hostname": "R1", "status": "up" }</pre>	<pre>xml <device> <hostname>R1</hostname> <status>up</status> </device></pre>

Formato JSON	Formato XML
Pares clave-valor Llaves { } para el objeto Más corto y compacto Fácil de leer y escribir Soporta string, número, booleano, null, arreglo y objeto REST APIs (Cisco DNA Center, APIC)	Más largo, porque cada dato repite el nombre de la etiqueta Más verboso, pero muy descriptivo Todo es texto; los tipos se definen con esquemas (XSD) NETCONF y sistemas heredados